Challenge task 4 – Spike report

Student Name: Douha Assafiri

Student ID: 104422187

Figma Prototype: https://www.figma.com/proto/kJoh9WUvMHeuMc4tB5MoRV/Scout?node-id=2-2&p=f&viewport=719%2C589%2C0.11&t=kydN9cvRU78IItjL-1&scaling=min-zoom&content-scaling=fixed&starting-point-node-id=2%3A2&show-proto-sidebar=1&page-id=0%3A1

GitHub Repository: https://github.com/douha-assafiri/a1-task-4-custom-app-development-douha-assafiri

Video Demonstration: https://youtu.be/LOs8RMGnoIc

Level 1: Design

1. Introduction

Scout is an Android application that enables users to discover and learn about Australian flora and fauna, and log their own sightings. It uses the iNaturalist public API as its primary data source and stores user data locally on the device.

This report covers all three levels of Challenge Task 4. The design addresses all three unit learning outcomes: ULO 1 by applying an understanding of mobile-specific constraints such as network dependency, screen orientation, and runtime permissions; ULO 2 by designing an interface that accounts for hardware restrictions including screen size through adaptive portrait and landscape layouts; and ULO 3 by planning and implementing the use of standard Android libraries including Jetpack components throughout the application.

2. Investigation: Accessing Online Data in Android

A key aspect of Scout's development is fetching species data from a remote API. This section investigates the iNaturalist API and compares two Android networking libraries: Retrofit and Volley.

2.1 The iNaturalist API

The iNaturalist API is a free, publicly accessible REST API that does not require an API key for read-only access and allows up to 100 requests per minute. It returns species data including common names, scientific names, photos, conservation status, and geographic observations. It supports a locale parameter that returns common names in the requested language, directly enabling Scout's localisation for Arabic and French alongside English.

2.2 Library Comparison: Retrofit vs Volley

Two libraries were evaluated for handling network requests in Scout, focusing on coroutine support, architecture compatibility, testability, and ease of use.

Retrofit (Square):

- Advantages: Type-safe API interfaces via annotations, native Kotlin Coroutines support, integrates cleanly with MVVM, automatic JSON parsing via Gson, widely adopted in industry.
- Disadvantages: Requires more initial setup and understanding of interface-based design patterns.

Volley (Google):

- Advantages: Simple request queue model, easy to get started, suitable for straightforward single requests.
- Disadvantages: Callback-based rather than coroutine-friendly, less structured, harder to unit test, less actively maintained.

2.3 Decision

Retrofit was selected for Scout for three reasons. First, its native Kotlin Coroutines support aligns directly with the unit's concurrency requirements. Second, its interface-based design integrates naturally with the MVVM architecture. Third, it is better suited for unit testing as network calls can be mocked through the interface.

Volley's callback-based approach would add complexity when handling concurrent requests such as loading species lists and images simultaneously.

2.4 Data Storage Strategy

Scout uses two storage mechanisms. API responses provide live species data when the device is online. Favourites and sightings are stored persistently in a local Room database, allowing users to access saved species and personal logs without an internet connection. This separation of concerns reflects ULO 1's requirement to design software with awareness of the mobile environment's connectivity constraints.

3. App Design

3.1 Vision Statement

Scout aims to make the discovery of Australian plants and animals accessible, educational, and engaging for a broad audience including casual nature enthusiasts, tourists visiting Australia, students, and citizen scientists. Users can browse and search species fetched from the iNaturalist API, log their own sightings with photos and location, and save favourites for offline access.

3.2 User Stories

- As a nature enthusiast, I want to browse Australian plants and animals so that I can learn about local species.
- As a user, I want to search for a specific species so that I can quickly find information about it.
- As a user, I want to log a sighting with a photo and location so that I can keep a personal record.
- As a user, I want to save favourite species so that I can access them offline.
- As a tourist, I want to see species near my location so that I know what I might spot.
- As a user, I want to understand a species' conservation status so that I appreciate its ecological significance.
- As a user, I want to take a photo or choose from my gallery when logging a sighting so that I have flexibility in how I capture evidence.
- As a non-English speaker, I want the app to display species names and UI in my native language.

3.3 Use Cases

The following use cases describe the key interactions between a user and Scout.

Use Case 1: Browse and View a Species	
Actor	User
Precondition	App is open and device has an internet connection.
Main Flow	1. User opens the Explore screen. 2. User taps the Plants or Animals tab. 3. App fetches and displays a list of species from the iNaturalist API. 4. User taps on a species card. 5. App navigates to the Species Detail screen showing name, scientific name, conservation status, habitat, and a Did You Know fact.

Alternative Flow	If the device has no internet connection, the species list fails to load and an error Snackbar with a Retry action is shown.
Postcondition	User has viewed the species detail.

Use Case 2: Log a Sighting	
Actor	User
Precondition	App is open. Camera or gallery permission may be requested.
Main Flow	1. User taps the Log tab. 2. User taps Camera or Upload to add a photo. 3. User selects a species from the autocomplete list. 4. App fills in species name and category automatically. 5. User taps Use Current Location. 6. User taps Log Sighting. 7. App saves the sighting to the Room database and shows a Snackbar confirmation.
Alternative Flow	If camera permission is denied, a Snackbar informs the user. If no species is selected from the autocomplete, a validation error is shown.
Postcondition	Sighting is saved locally and visible in the sightings history.

Use Case 3: Save a Favourite Species	
Actor	User
Precondition	User is on the Species Detail screen.
Main Flow	1. User taps the heart icon on the Species Detail screen. 2. App saves the species to the Room favourites table. 3. A Toast confirms the species was added to favourites. 4. Species appears in the Favourites screen.
Alternative Flow	If the species is already saved, tapping the heart removes it from favourites and shows a Toast confirmation.
Postcondition	Species is stored in Room and accessible offline from the Favourites screen.

3.4 Prototype

A high-fidelity interactive prototype was developed in Figma covering both portrait and landscape orientations across all major screens. The prototype includes clickable connections linking all core user flows including species browsing, detail view, logging a sighting with camera and gallery flows, favourites management, and profile.

Prototype link:
https://www.figma.com/proto/kJoh9WUvMHeuMc4tB5MoRV/Scout?node-id=2-2&p=f&viewport=719%2C589%2C0.11&t=kydN9cvRU78IltjL-1&scaling=min-zoom&content-scaling=fixed&starting-point-node-id=2%3A2&show-proto-sidebar=1&page-id=0%3A1

Figure 1: Figma prototype overview: all screens

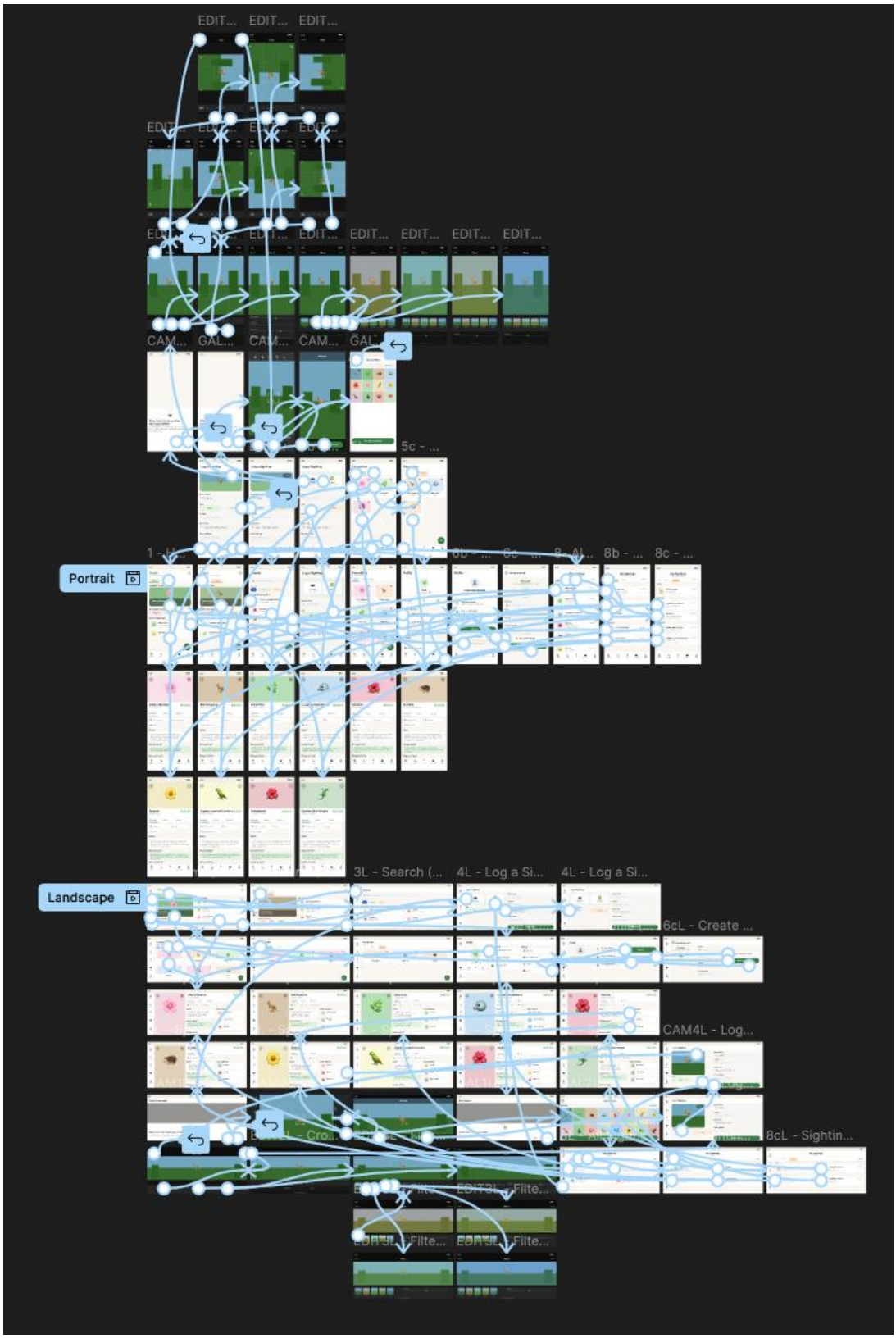
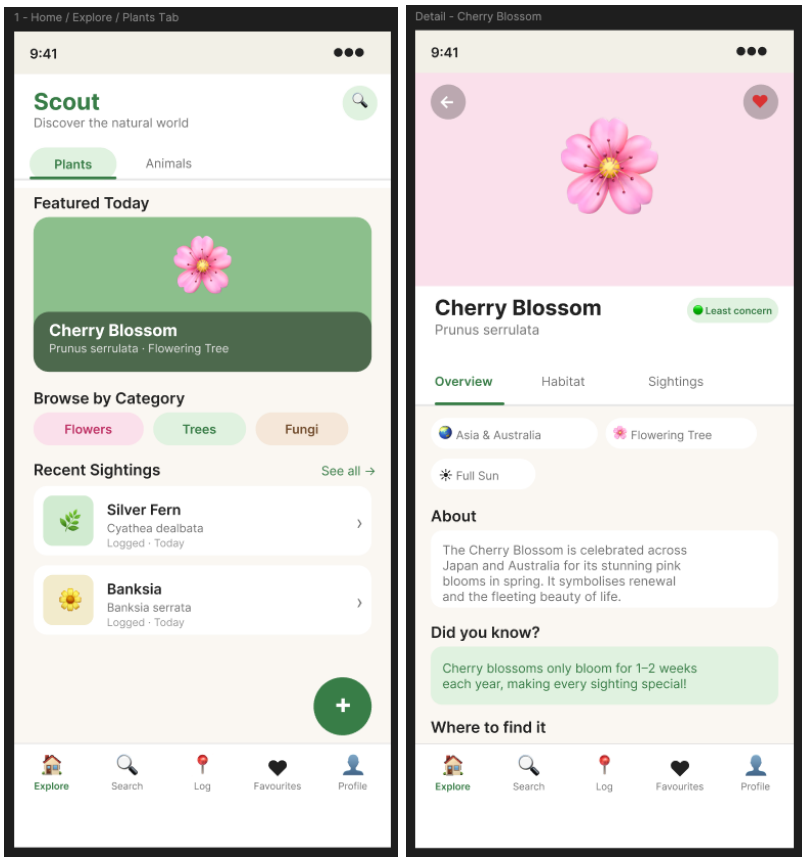


Figure 2: Explore screen and Species Detail screen



4. User Study

4.1 Methodology

The Figma prototype link was shared with three participants who were asked to navigate freely and then provide feedback through synchronous interviews. Participants are referred to as P1, P2, and P3 to protect anonymity.

4.2 Participants

- P1: Adult Australian resident with a general interest in nature and familiarity with mobile apps.
- P2: Adult who recently arrived in Australia from another country, unfamiliar with local wildlife, interested in learning about the local environment.
- P3: Adult with a background in software and mobile apps, familiar with Android design conventions.

4.3 Findings: Positive Feedback

- All three participants described Scout as straightforward and easy to use.
- P1 noted Scout's potential as a tool for researchers, and as a platform for collecting biodiversity data. P1 described it as educational and fun, noting it encourages people to go outside.

- P2 expressed that Scout would help her learn about Australian wildlife she had never encountered before, and appreciated being able to explore species she could not find elsewhere.
- P3 found the camera flow clean and appreciated the side-by-side Camera and Upload options.

4.4 Findings: Suggestions for Improvement

- Community sightings (P1): Suggested showing recent sightings by other users on each species detail screen. Noted as future scope.
- App title (P1): Suggested ‘Discover Australia’s Plants and Animals’ to better communicate the app’s purpose.
- Species name entry (P1): The Log a Sighting form should offer a searchable dropdown of known species.
- Conservation status (P1): The term ‘Least Concern’ was unclear. A popup explaining the IUCN scale was suggested.
- Map view (P2): Showing sightings on an interactive map was suggested as more informative than text.
- Contact the logger and professional inquiry (P2): Noted as valuable future features.
- Photo standards and star reviews (P2): Minimum photo quality standards and a rating system were suggested.
- Rank visibility (P2): The user rank on the Profile screen should be more prominent.
- Conservation status colour coding (P3): Suggested red for Endangered, green for Least Concern.
- Date field default (P3): The date field should auto-fill with the current date and time.
- Favourites search (P3): A search within the Favourites screen would be useful as the list grows.
- Photo editing scope (P3): Photo editing may be beyond what is needed and could be simplified.

4.5 Changes Made in Response to Feedback

- The app title was updated to Discover Australia’s Plants and Animals.
- A conservation status tooltip was added to the Species Detail screen.
- The species name field was updated to use autocomplete with a dropdown of known species.
- The date and time field auto-fills with the current date and time by default.
- The Profile screen was updated with Level number and a guide to how many sightings until next level.
- Community features are noted as valuable future scope beyond the current project.

Level 2: App

5. Application Architecture

5.1 Architecture Pattern: MVVM

Scout follows the Model-View-ViewModel (MVVM) architecture pattern throughout. This choice was made for three reasons. First, MVVM cleanly separates UI logic (Fragment) from data logic (ViewModel), making the codebase easier to maintain and test. Second, it integrates naturally with Android's Jetpack Lifecycle components, specifically LiveData and ViewModel. Third, it is the architecture pattern recommended by Google for Android development, representing current practice in the industry.

The three layers are implemented as follows:

- Model: The data/ package contains all API models (TaxonResult, TaxaResponse, ConservationStatus, TaxonPhoto), Room entities (FavouriteEntity, SightingEntity), DAOs (FavouriteDao, SightingDao), and Repositories (SpeciesRepository, FavouriteRepository, SightingRepository).
- ViewModel: One ViewModel per Fragment, each extending AndroidViewModel or ViewModel. All ViewModels hold MutableLiveData and call repositories via Kotlin Coroutines using viewModelScope.launch.
- View: Fragments observe LiveData via viewLifecycleOwner and update the UI. No business logic exists in Fragment classes.

5.2 Repository Pattern

A Repository layer sits between the ViewModels and the data sources. This abstraction means ViewModels do not know whether data comes from the API or Room, making it straightforward to swap or extend data sources in future. Three repositories are implemented:

Repository	Data Source	Used By
SpeciesRepository	INaturalistApi (Retrofit)	Explore, Search, Detail, Log
FavouriteRepository	FavouriteDao (Room)	Detail, Favourites, Profile
SightingRepository	SightingDao (Room)	Log, Detail, Profile, Sightings

5.3 Navigation

Scout uses Jetpack Navigation Component with a single NavHostFragment in MainActivity. Bottom navigation with 5 tabs (Explore, Search, Log, Favourites, Profile) provides the primary navigation. A FAB on the main screen also navigates directly to the Log Sighting screen. Fragment-to-fragment navigation passes arguments (taxonId: Long) using Safe Args, ensuring type safety. The navigation graph defines all destinations and actions, eliminating manual Fragment transactions throughout the app.

6. Fragments and Features

Scout implements 7 distinctly different fragments:

Fragment	Type	Key Features
ExploreFragment	RecyclerView + other views	Featured hero card, Plants/Animals tabs, category chips, recent sightings, swipe-to-refresh
SearchFragment	RecyclerView + search bar	SearchView with 500ms debounce, filter chips, trending default results
LogSightingFragment	Form: takes data in	Camera/gallery with ActivityResultContracts, species autocomplete, GPS location, validation, Snackbar confirmation
SpeciesDetailFragment	Detail: no data input	Hero image, conservation badge, Overview/Habitat/Sightings tabs, similar species, heart toggle with Toast
FavouritesFragment	RecyclerView only	2-column grid, All/Plants/Animals filter, Room database source
ProfileFragment	Stats + activity	Avatar, editable name, level system (5 levels), recent sightings, SharedPreferences name storage
SightingsFragment	Filtered list	Full sightings history, category filter, clear-all, Room database source

7. Key Technical Decisions

7.1 Kotlin Coroutines and LiveData

All API calls and database operations use Kotlin Coroutines via viewModelScope.launch, ensuring network and I/O operations never block the main thread. Results are exposed to fragments via LiveData, which is lifecycle-aware and prevents UI updates on destroyed fragments. This combination directly satisfies ULO 3's requirement for standard Android library usage, and represents current best practice for asynchronous Android development.

7.2 Room Database

Room was chosen over SharedPreferences for storing favourites and sightings because the data has multiple fields of different types (Long taxonId, String commonName, String photoUrl, Double latitude, Double longitude, Long timestamp, etc.), which is better suited to a relational database than key-value storage. Room

provides compile-time SQL verification, LiveData-returning queries that automatically notify the UI when data changes, and a clean DAO abstraction. Two tables are implemented: favourites and sightings.

7.3 Coil Image Loading

Coil was selected over Glide or Picasso because it is written in Kotlin, uses Coroutines natively, and integrates with Android's lifecycle. It handles caching, placeholder images, and error fallbacks automatically, which is important for Scout as species photos are loaded from remote iNaturalist URLs throughout the app.

7.4 KSP over KAPT

During development, Kotlin Symbol Processing (KSP) was used instead of KAPT for Room annotation processing. KSP is Kotlin-first, builds significantly faster than KAPT, and is Google's recommended approach for new Android projects.

7.5 Error Handling

Every API-calling ViewModel wraps network calls in try/catch blocks and posts error messages to a _error LiveData. Every fragment that calls the API observes this LiveData and shows a Snackbar with a Retry action. NetworkUtils.isOnline() exists as a proactive connectivity check utility. This ensures Scout handles the case where the device cannot connect to the API, as required by the Master-Detail brief.

7.6 Input Validation

LogSightingFragment implements a validateForm() method that checks two required fields: species name must not be blank, and a species must be selected from the autocomplete (ensuring taxonId and category are populated). Errors are displayed inline using TextInputLayout error messages. The form cannot be submitted until validation passes, ensuring data integrity in the Room sightings table.

7.7 Camera and Runtime Permissions

LogSightingFragment uses ActivityResultContracts.RequestPermission() for camera permission, ActivityResultContracts.TakePicture() for capturing photos, and ActivityResultContracts.GetContent() for gallery selection. Permissions are requested at the point of use, following Android's runtime permission model and the principle of least privilege introduced in Android 6.0. This reflects ULO 1's distinction between mobile and desktop environments.

7.8 Screen Size and Orientation (ULO 2)

Scout was designed for both portrait and landscape orientations. The res/layout-ldrtl/ directory provides the RTL-mirrored species card layout. These decisions directly address ULO 2's requirement to design for hardware-imposed constraints including screen size.

8. Libraries and Dependencies

The following libraries were used in Scout:

Library	Version	Purpose
Retrofit 2 + Gson converter	2.9.0	REST API calls to iNaturalist
Kotlin Coroutines (Android)	1.7.3	Async operations, non-blocking API/DB calls
AndroidX Lifecycle (ViewModel + LiveData)	2.7.0	MVVM architecture, lifecycle-aware data
Room (runtime + ktx + KSP compiler)	2.8.4	Local database for favourites and sightings
Jetpack Navigation Component	2.9.0	Fragment navigation, Safe Args
Coil	2.5.0	Async image loading with caching
RecyclerView	1.3.2	Species lists and favourites grid
SwipeRefreshLayout	1.1.0	Pull-to-refresh on Explore screen
Material Components	1.14.0	UI components (chips, cards, bottom nav, FAB)
ConstraintLayout	2.2.1	Flexible XML layouts

Figure 3: Log a Sighting screen and Camera permission dialog

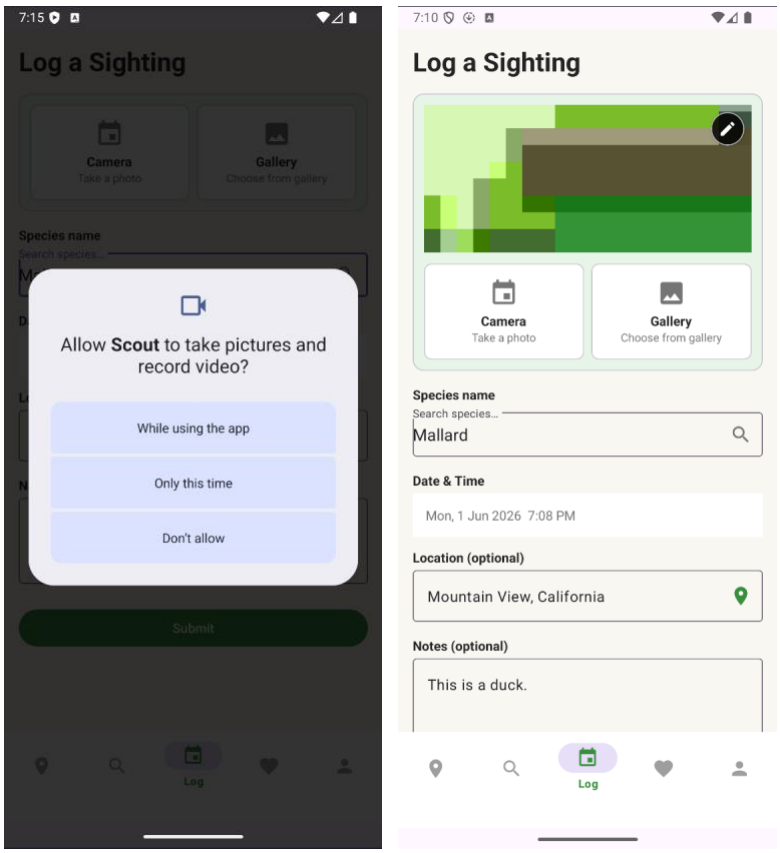
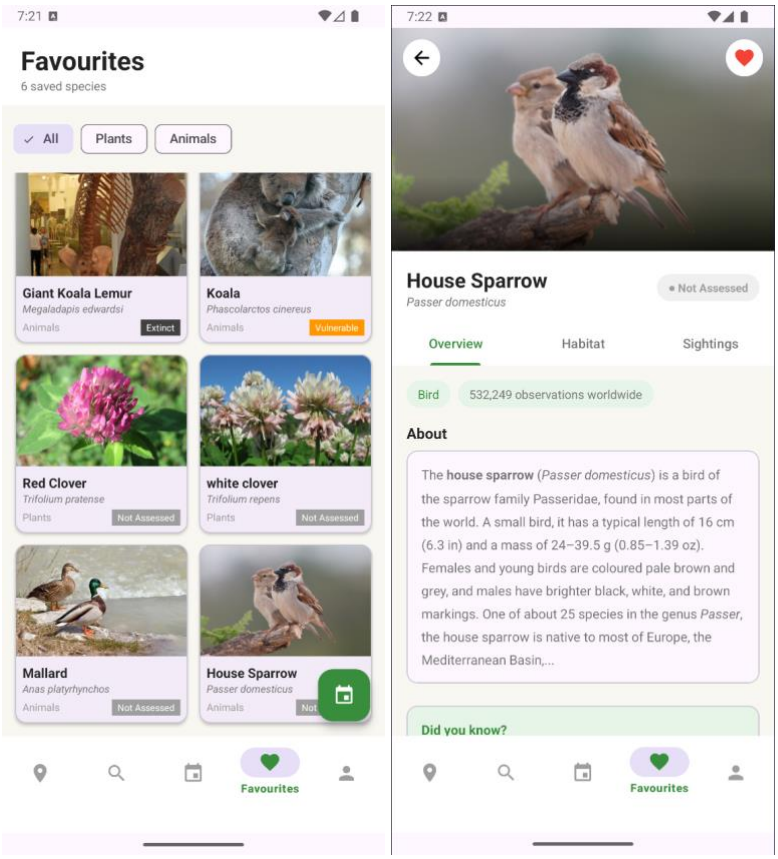


Figure 4: Favourites grid and Species Detail screen



Level 3: Research Report

9. RTL Localisation and Inclusive Design in Android

9.1 Research Question

How can an Android nature exploration app be designed and implemented to provide an inclusive, native-quality experience for Arabic-speaking users through Right-to-Left (RTL) localisation?

9.2 Motivation

This research question arose directly from the Scout user study. P2, a participant who had recently arrived in Australia from an Arabic-speaking country, was one of Scout's most enthusiastic users. She expressed that Scout could help people from her background learn about Australian wildlife that they had never encountered in their home countries. This highlighted a real user need: Scout's value for international visitors and new residents is significantly higher when the app respects their language and reading direction.

Arabic is spoken by approximately 422 million people worldwide and is the fifth most spoken language globally (Ethnologue, 2023). Despite this, only a small fraction of mobile applications provide genuine Arabic RTL support beyond simple text translation. According to Volchek (2021), just 1.1% of websites are available in Arabic, meaning Arabic speakers must navigate most of the digital content in a second language. Supporting RTL is therefore a meaningful accessibility intervention.

9.3 Literature Review

9.3.1 RTL Layout Mirroring

Awwad (2016) conducted a study on automated bidirectional localisation testing for Android applications, finding that RTL localisation involves far more than text direction. Their research identified that a compliant Arabic Android application must mirror directional UI elements including navigation arrows, progress indicators, and layout anchors, while preserving non-directional elements such as numbers, media playback controls, and brand logos. Awwad's work introduced an automated testing framework for detecting bidirectional defects, concluding that manual testing alone is insufficient for complex RTL interfaces.

Google's Material Design Bidirectionality guidelines (Google, 2024) formalise these principles for Android. The guidelines specify that an RTL layout is the mirror image of its LTR counterpart, and that the Android system handles layout mirroring automatically when android:supportsRtl="true" is declared in the manifest and all layout attributes use start/end rather than left/right. Specific elements that must not be mirrored include clocks, phone numbers, and media controls, as these communicate concepts rather than direction.

9.3.2 Cultural Dimensions of Arabic UI Design

Nejati et al. (2024) investigated the significance of culturally tailored digital products for Arabic-speaking users, using Hofstede's cultural dimensions framework and a combination of A/B testing, surveys, and interviews. Their findings demonstrated that ease of understanding varies for native Arabic speakers, and that feature

preferences differ based on nationality. Importantly, they found that cultural relevance in UI design, encompassing layout, colour, and language, has a significant relationship with users' behavioural intention to use an application. This supports the decision to implement RTL in Scout as a genuine UX improvement rather than a superficial translation exercise.

UX research in the Arab world has also highlighted that navigation and interaction patterns are culturally influenced. Bilingual Arabic/English mobile applications are common in Gulf and Middle Eastern contexts, and users frequently switch between languages within a session. This informed Scout's decision to detect the device locale at runtime rather than requiring the user to manually select a language.

9.3.3 Android Internationalisation Practices

Liu et al. (2022) studied automated Android app internationalisation, finding that many developers fail to implement complete i18n due to the labour-intensive nature of the task. Their research proposed an automated approach (Androi18n) using neural networks to assist with term translations. The study evaluated results across five languages and found that reliable translations could be generated from existing app corpora. For Scout, this research highlighted the importance of separating all user-visible strings into strings.xml resource files from the outset, a practice that was followed throughout development and that made the Arabic and French translations straightforward to add.

Warburton (2023) documented the localisation of a university campus phone app, noting that the Functionalist approach to translation, focusing on the function of each string in its context rather than literal translation, is most appropriate for mobile app localisation. Technical and linguistic challenges identified included string length variation between languages, and the need for translators to understand UI context. Scout encountered the string length challenge with Arabic, where some labels are significantly longer in Arabic than in English, requiring adequate padding in layout files.

9.3.4 RTL and Ergonomics

Recent UX research in Arabic-first markets has questioned whether strict layout mirroring always produces the optimal experience. Agarwal (2026) found that in testing across eight client apps, users consistently preferred the primary call-to-action button to remain centrally anchored or bottom-right even in Arabic mode, due to the ergonomics of right-handed single-handed phone use. Apps that selectively broke strict RTL mirroring for the primary action achieved a 22% higher conversion rate. This nuanced finding suggests that RTL implementation benefits from user testing rather than mechanical mirroring, and informs future iterations of Scout's design.

9.4 Implementation in Scout

9.4.1 What Was Implemented

- android:supportsRtl="true" added to the application element in AndroidManifest.xml, enabling Android's automatic layout mirroring system.
- All fragment XML layouts audited to replace left/right attributes with start/end equivalents (marginStart, marginEnd, paddingStart, paddingEnd, layout_constraintStart_toStartOf), ensuring physical layout mirroring works correctly in RTL.

- res/layout-ldrtl/item_species_card.xml created as an RTL-specific layout variant. Android automatically loads this resource when the device is in RTL mode. In this variant, the species photo is positioned on the right and text on the left, matching the Arabic reading direction.
- LocaleUtils.kt created as a centralised utility providing apiLocale() (returns the iNaturalist API locale code based on device language) and isRtl(context) (returns whether the current configuration is RTL). This utility is used throughout the app.
- SpeciesRepository detects the device language at runtime via LocaleUtils.apiLocale() and passes it to every iNaturalist API call. When the device is set to Arabic, the API returns species common names and Wikipedia summaries in Arabic.
- res/values-ar/strings.xml containing full Arabic translations of all UI strings, including navigation labels, form fields, error messages, level titles, and confirmations.
- res/values-fr/strings.xml containing full French translations of all UI strings.
- Html.fromHtml() used in SpeciesDetailFragment to correctly render HTML-formatted species descriptions (bold, italic tags from Wikipedia) in all languages including Arabic.

9.4.2 Visual Demonstration

Figure 5: Scout in English (LTR) vs Arabic (RTL): Explore screen

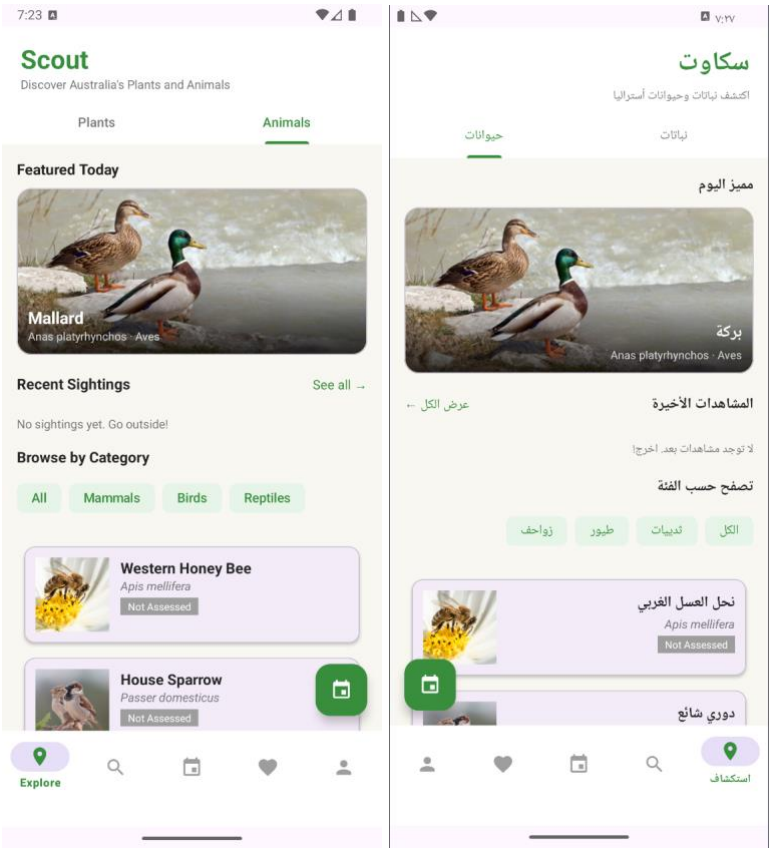
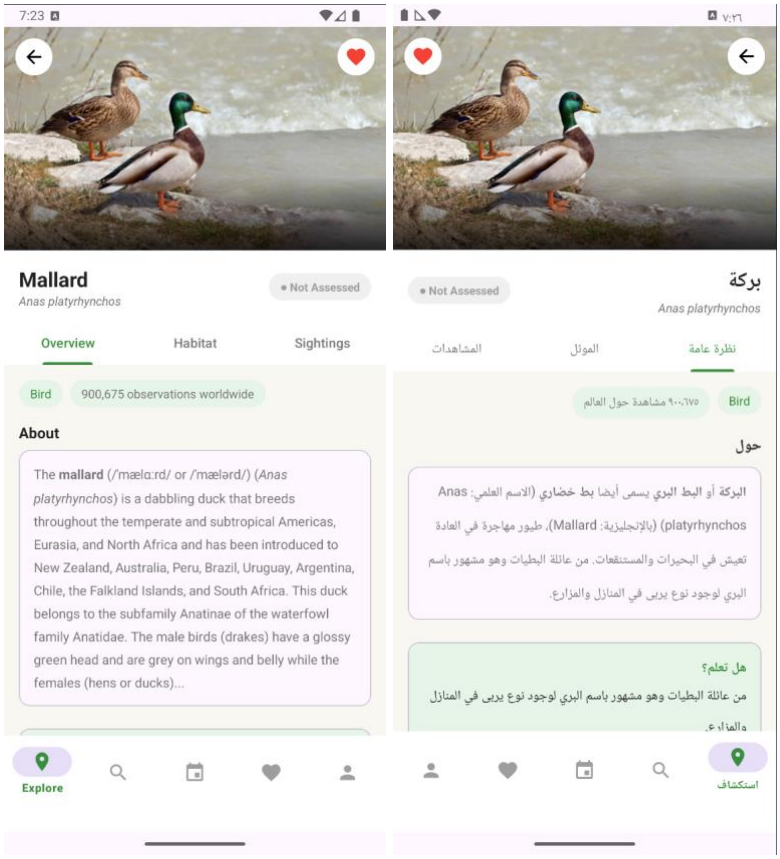


Figure 6: Scout in English (LTR) vs Arabic (RTL): Species Detail



9.5 Evaluation

The RTL implementation in Scout demonstrates a meaningful improvement in inclusivity for Arabic-speaking users beyond what simple text translation provides:

- Layout mirroring: The entire app layout physically mirrors in Arabic, with navigation arrows pointing in the correct direction, text right-aligned, and UI elements in their expected positions for RTL readers.
- Content localisation: Species common names and Wikipedia summaries are returned in Arabic by the iNaturalist API when the device language is Arabic, meaning the informational content of the app is localised.
- Species card RTL variant: The custom ldrtl layout variant for the species card demonstrates a deliberate design decision rather than relying solely on automatic mirroring, ensuring the card layout is optimised for Arabic reading direction.
- Known limitation: iNaturalist does not index Arabic common names for all species, meaning some species may still display their scientific (Latin) name or English common name when the API does not have an Arabic equivalent. This is an API data limitation rather than an implementation limitation.

The implementation aligns with the best practices outlined in the literature. Following Awwad (2016), non-directional elements (numbers, timestamps, media controls) were not mirrored. Following Google's Material Design guidelines (2024), start/end layout attributes were used throughout rather than left/right. The locale detection

approach follows the pattern recommended by Android Developers (2024), using the device configuration rather than requiring manual language selection.

9.6 Conclusions

The research question “How can an Android nature exploration app provide an inclusive experience for Arabic-speaking users through RTL localisation?” was answered through a combination of literature review and a practical implementation in Scout.

The literature confirmed that genuine RTL support requires more than text translation: it requires layout mirroring, locale-aware content, and attention to non-directional elements. The implementation in Scout addressed all of these dimensions. The API locale parameter provided a particularly elegant solution, returning Arabic species names without requiring Scout to maintain its own Arabic species database.

Future work could include a formal usability evaluation of the Arabic version with native Arabic-speaking participants, addressing the ergonomic considerations raised by Agarwal (2026) regarding primary action button placement in RTL mode.

9.7 References

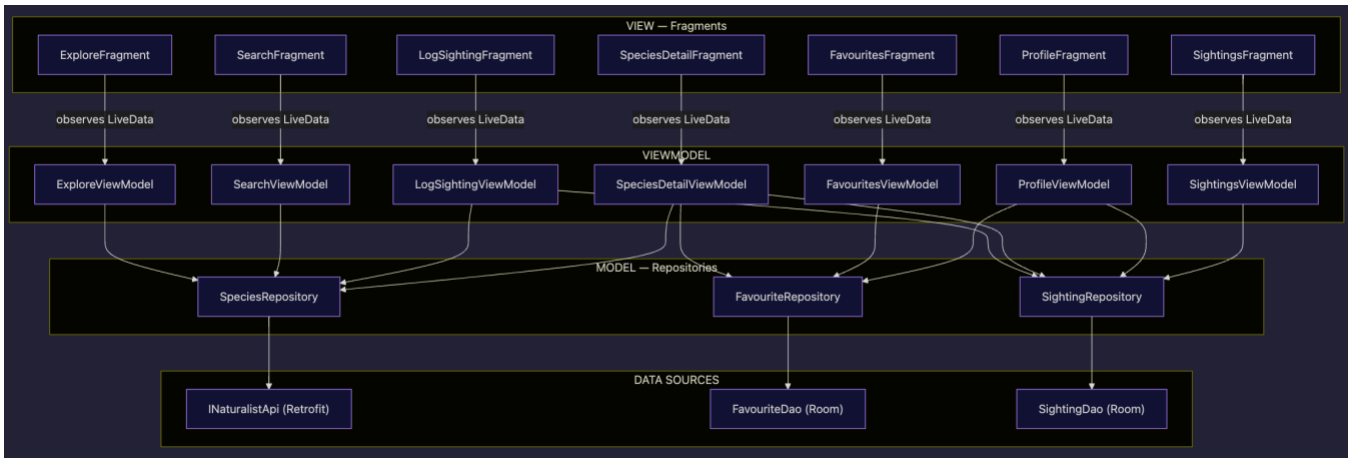
- Ayyal Awwad, A. M., & Slany, W. (2016). Automated Bidirectional Languages Localization Testing for Android Apps with Rich GUI. Mobile Information Systems, 2016, 1–13. https://doi.org/10.1155/2016/2872067
- Liu, P., Xia, Q., Liu, K., Guo, J., Wang, X., Liu, J., Grundy, J., & Li, L. (2023). Towards automated Android app internationalisation: An exploratory study. *Journal of Systems and Software, 197*, 111559. https://doi.org/10.1016/j.jss.2022.111559
- Agarwal, G. (2026). UX design Bahrain Arabic 2026: RTL mobile app experience guide. https://gaurav.imapro.in/ux-design-bahrain-arabic-2026-rtl-mobile-app-experience
- Google. (2024). Bidirectionality - Material Design. https://material.io/design/usability/bidirectionality.html
- Google. (2024). Support different languages and cultures - Android Developers. https://developer.android.com/training/basics/supporting-devices/languages
- Nejati, B., Manchi, R. T., & Schofield, D. (2024). Incorporating Cross-Cultural Design into the User Interface. *The International Journal of Multimedia & Its Applications, 16*(5), 01-19. https://doi.org/10.5121/ijma.2024.16501
- Volchek, K. (2021). Mobile app design for right-to-left languages (Arabic RTL localisation). https://kristi.digital/shots/mobile-app-design-for-right-to-left-languages-arabic-language
- Warburton, K. (2023). Localizing a phone app: Challenges and reflections. *Digital Translation, 10(1),* 88–120. https://doi.org/10.1075/dt.22010.war

Appendix

A. MVVM Architecture Diagram

The following diagram illustrates Scout's MVVM architecture, showing how each Fragment connects to its ViewModel, which calls Repositories, which in turn access the API or Room database.

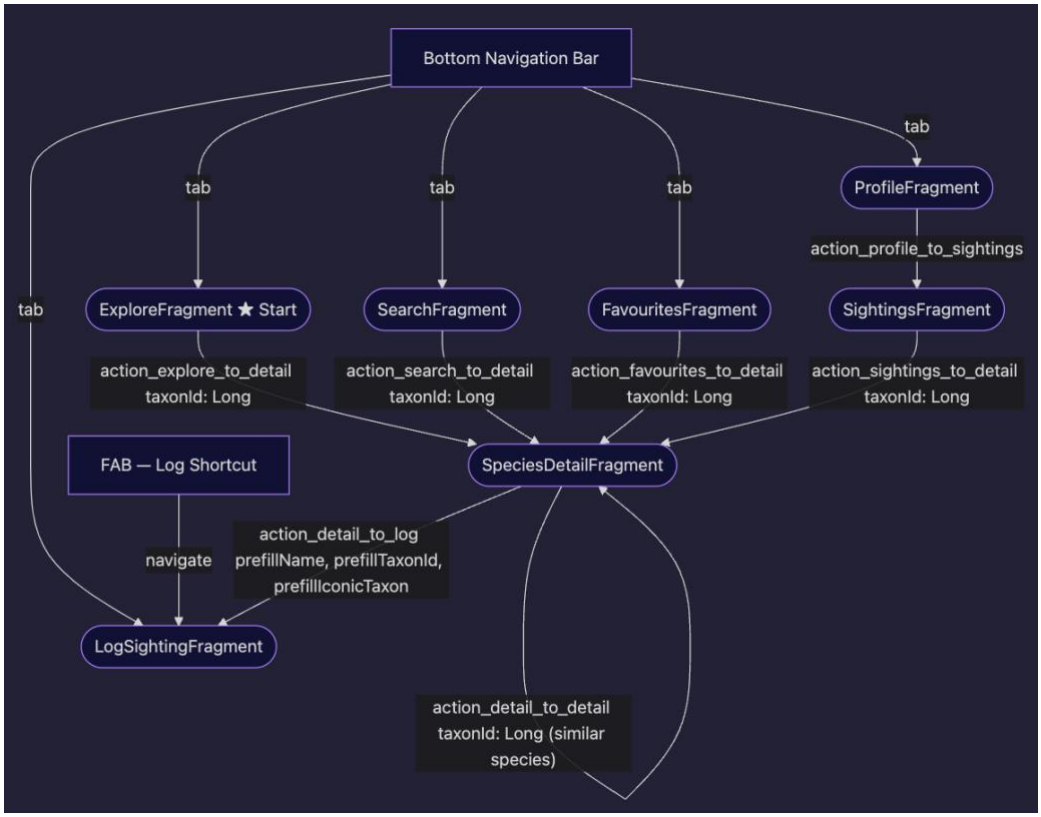
Figure A1: MVVM architecture diagram



B. Navigation Graph

The following diagram shows all 7 Fragment destinations and the navigation actions between them.

Figure B1: Navigation graph diagram



C. Room Database Schema

The following diagram shows the two Room database tables: favourites and sightings, with all fields and their types.

Figure C1: Room database schema diagram

FAVOURITES			
Long	taxonId	PK	iNaturalist taxon ID
String	commonName		Preferred common name
String	scientificName		Latin name
String	photoUrl		nullable - medium photo URL
String	conservationStatus		nullable - e.g. LC, EN, CR
String	category		Plants or Animals
Long	savedAt		epoch ms, default now()

SIGHTINGS			
Int	id	PK	auto-generated
Long	taxonId		nullable - iNaturalist taxon ID
String	speciesName		display name
String	category		Plant or Animal
String	photoPath		nullable - local file path or URI
Double	latitude		nullable
Double	longitude		nullable
String	locationName		nullable - geocoded place name
String	notes		nullable - user notes
Long	loggedAt		epoch ms, default now()

D. Package Structure

The complete package structure of the Scout Android project:

```
app/src/main/
├── AndroidManifest.xml
├── java/com/example/scout/
│   ├── MainActivity.kt
│   ├── data/
│   │   ├── api/
│   │   │   ├── ApiClient.kt
│   │   │   ├── INaturalistApi.kt
│   │   │   └── models/
│   │   │       ├── ConservationStatus.kt
│   │   │       ├── TaxaResponse.kt
│   │   │       ├── TaxonDetailResponse.kt
│   │   │       ├── TaxonPhoto.kt
│   │   │       └── TaxonResult.kt
│   │   ├── db/
│   │   │   ├── FavouriteDao.kt
│   │   │   ├── ScoutDatabase.kt
│   │   │   ├── SightingDao.kt
│   │   │   └── entities/
│   │   │       ├── FavouriteEntity.kt
│   │   │       └── SightingEntity.kt
│   │   └── repository/
│   │       ├── FavouriteRepository.kt
│   │       ├── SightingRepository.kt
│   │       └── SpeciesRepository.kt
│   ├── ui/
│   │   ├── detail/
│   │   │   ├── SpeciesDetailFragment.kt
│   │   │   └── SpeciesDetailViewModel.kt
│   │   ├── explore/
│   │   │   ├── ExploreFragment.kt
│   │   │   ├── ExploreViewModel.kt
│   │   │   └── SpeciesAdapter.kt
│   │   ├── favourites/
│   │   │   ├── FavouritesAdapter.kt
│   │   │   ├── FavouritesFragment.kt
│   │   │   └── FavouritesViewModel.kt
│   │   ├── log/
│   │   │   ├── LogSightingFragment.kt
│   │   │   └── LogSightingViewModel.kt
│   │   ├── profile/
│   │   │   ├── ProfileFragment.kt
│   │   │   └── ProfileViewModel.kt
```

```
│   │   ├── search/
│   │   │   ├── SearchFragment.kt
│   │   │   └── SearchViewModel.kt
│   │   └── sightings/
│   │       ├── SightingsAdapter.kt
│   │       ├── SightingsFragment.kt
│   │       └── SightingsViewModel.kt
│   └── utils/
│       ├── Extensions.kt
│       ├── ImageUtils.kt
│       ├── LevelSystem.kt
│       ├── LocaleUtils.kt
│       └── NetworkUtils.kt
└── res/
    ├── color/nav_item_color.xml
    ├── drawable/ (18 vector/shape files)
    ├── layout/
    │   ├── activity_main.xml
    │   ├── fragment_explore.xml
    │   ├── fragment_favourites.xml
    │   ├── fragment_log_sighting.xml
    │   ├── fragment_profile.xml
    │   ├── fragment_search.xml
    │   ├── fragment_sightings.xml
    │   ├── fragment_species_detail.xml
    │   ├── item_favourite_card.xml
    │   ├── item_recent_sighting.xml
    │   ├── item_sighting_row.xml
    │   ├── item_similar_species.xml
    │   └── item_species_card.xml
    ├── layout-ldrtl/
    │   └── item_species_card.xml
    ├── menu/bottom_nav_menu.xml
    ├── navigation/nav_graph.xml
    ├── values/
    │   ├── colors.xml
    │   ├── plurals.xml
    │   ├── strings.xml
    │   └── themes.xml
    ├── values-ar/strings.xml
    ├── values-fr/strings.xml
    └── xml/ (backup_rules, file_paths)
```